

Flutter

Работа с текстом в flutter. TextEditingController и ListView



TextEditingController

TextEditingController - класс, который управляет текстом в TextField (поле для ввода данных) и других виджетах, позволяя:

- получать текущее значение текста
- изменять текст программно
- реагировать на изменения текста с помощью слушателей



TextEditingController

чтобы использовать TextEditingController, нужно создать его экземпляр.

```
final TextEditingController _controller = TextEditingController();
```

чтобы связать контроллер с текстовым полем, передаем его в параметр controller виджета TextField:

```
content: TextField(
  controller: _controller,
  decoration: InputDecoration(hintText: "Введите текст заметки"),
), // TextField
```

TextEditingController

чтобы получить текущее значение текста из контроллера, можно использовать свойство text. это значение можно использовать, например, при нажатии кнопки:

```
ElevatedButton(
  onPressed: () {
  String inputValue = _controller.text;
  // Используйте inputValue по вашему ус
  },
  child: Text('Отправить'),
), // ElevatedButton
```

можно изменять текст в TextField, устанавливая новое значение в контроллер:

```
_controller.text = 'Новое значение';
```



TextEditingController

также можно добавлять слушателей для отслеживания изменений текста. если нужно выполнять действия при каждом изменении, это будет как раз:

```
_controller.addListener(() {
  print('Текущий текст: ${_controller.text}');
});
```

не забываем удалить слушателя, когда он больше не нужен, чтобы избежать утечек памяти:

```
  @override
void dispose() {
  _controller.dispose();
  super.dispose();
}
```



пример работы 1

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(FirstExample());
}

class FirstExample extends StatelessWidget {
  const FirstExample({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('очистка текста')),
        body: FirstExampleWidget(),
      ), // Scaffold
    ); // MaterialApp
  }
}

class FirstExampleWidget extends StatefulWidget {
  const FirstExampleWidget({super.key});

  @override
  _FirstExampleWidgetState createState() => _FirstExampleWidgetState();
}
```

```dart
class _FirstExampleWidgetState extends State<FirstExampleWidget> {
  final TextEditingController _controller = TextEditingController();
  String _displayText = '';

  @override
  void initState() {
    super.initState();
    _controller.addListener(_updateDisplayText);
  }

  @override
  void dispose() {
    _controller.removeListener(_updateDisplayText);
    _controller.dispose();
    super.dispose();
  }

  void _updateDisplayText() {
    setState(() {
      _displayText = _controller.text;
    });
  }

  void _clearText() {
    _controller.clear();
  }

```

пример работы 1

```dart
54
55    @override
56    Widget build(BuildContext context) {
57      return Column(
58        children: [
59          TextField(
60            controller: _controller,
61            decoration: InputDecoration(
62              hintText: 'введите текст',
63            ), // InputDecoration
64          ), // TextField
65          ElevatedButton(
66            onPressed: _clearText,
67            child: Text('очистить текст'),
68          ), // ElevatedButton
69          Text('тут то что вы ввели: $_displayText'),
70        ],
71      ); // Column
72    }
73  }
74
```



пример работы 2

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(SecondExample());
}

class SecondExample extends StatelessWidget {
  const SecondExample({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('показ текста')),
        body: SecondExampleWidget(),
      ), // Scaffold
    ); // MaterialApp
  }
}

class SecondExampleWidget extends StatefulWidget {
  const SecondExampleWidget({super.key});

  @override
  _SecondExampleWidgetState createState() => _SecondExampleWidgetState();
}

class _SecondExampleWidgetState extends State<SecondExampleWidget> {
  final TextEditingController _controller = TextEditingController();
  String _displayText = '';
```

```dart
  @override
  void initState() {
    super.initState();
    _controller.addListener(_updateDisplayText);
  }

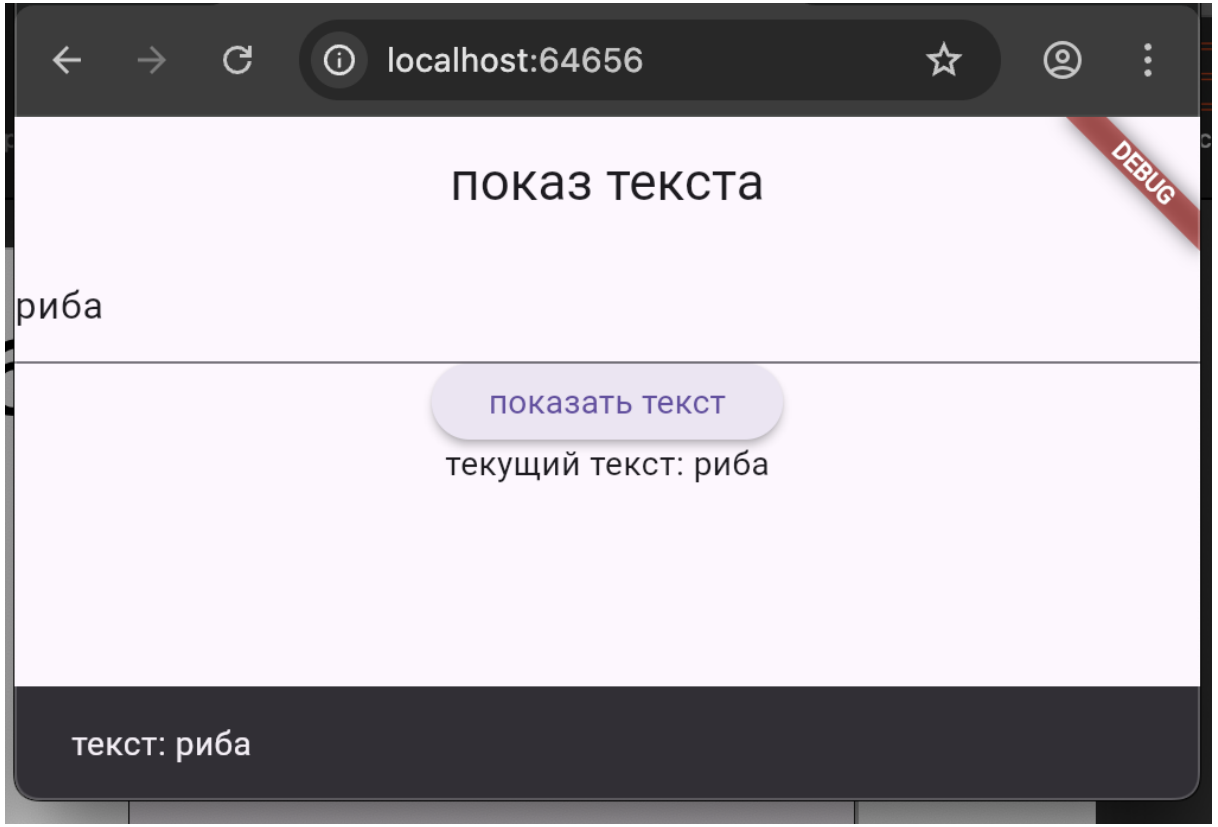
  @override
  void dispose() {
    _controller.removeListener(_updateDisplayText);
    _controller.dispose();
    super.dispose();
  }

  void _updateDisplayText() {
    setState(() {
      _displayText = _controller.text;
    });
  }

  void _showText() {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('текст: $_displayText')),
    );
  }
```

пример работы 2

```
57    @override
58    Widget build(BuildContext context) {
59      return Column(
60        children: [
61          TextField(
62            controller: _controller,
63            decoration: InputDecoration(
64              hintText: 'введите текст',
65            ), // InputDecoration
66          ), // TextField
67          ElevatedButton(
68            onPressed: _showText,
69            child: Text('показать текст'),
70          ), // ElevatedButton
71          Text('текущий текст: $_displayText'),
72        ],
73      ); // Column
74    }
75  }
```





ListView и ключи

ListView - элемент интерфейса, который показывает данные в виде прокручиваемого вертикального списка или таблицы,

- **children**: объект List<Widget>, который представляет список виджетов, добавляемых в ListView
- **scrollDirection**: устанавливает направление элементов. Представляет перечисление Axis, которое определяет две константы:
  - **Axis.horizontal**: устанавливает горизонтальный список - элементы располагаются слева направо (или справо налево)
  - **Axis.vertical**: устанавливает вертикальный список - элементы располагаются сверху вниз
- **padding**: устанавливает отступы элементов от границ ListView, представляет объект EdgeInsetsGeometry
- **reverse**: если равен true, располагает элементы в обратном порядке
- **physics**: задает параметры скроллинга с помощью объекта ScrollPhysics
- **controller** - управление прокруткой и прослушивание положения прокрутки

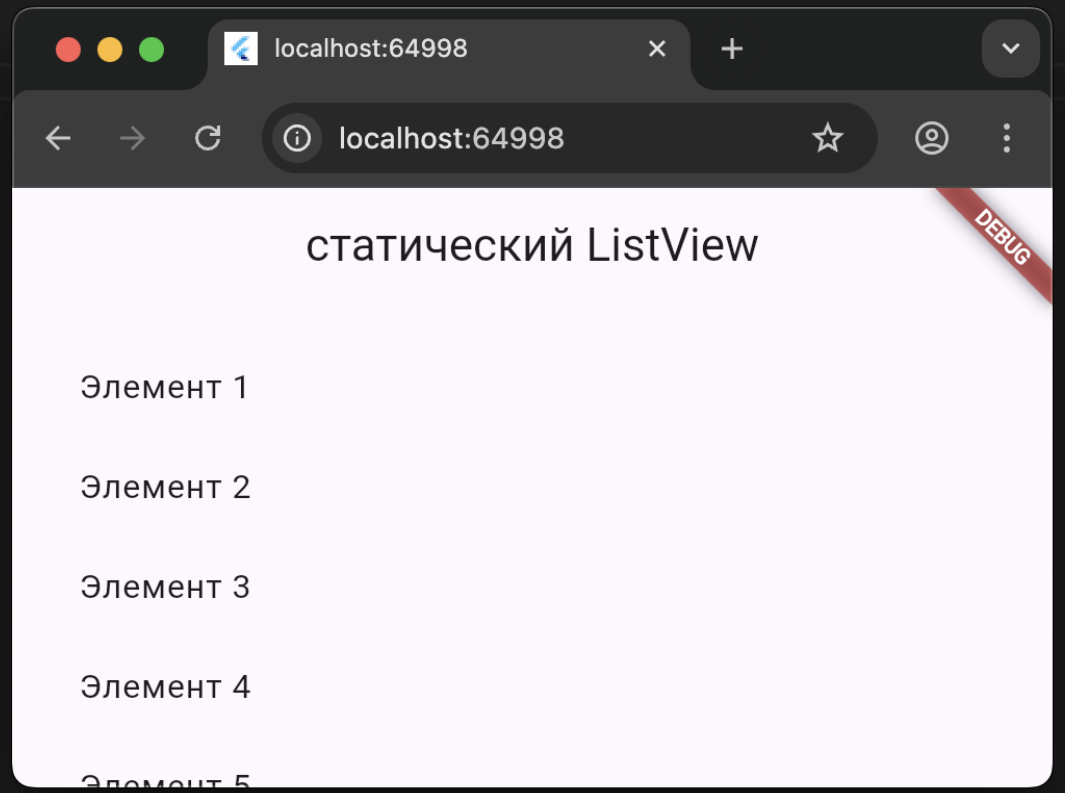
статический ListView

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(StaticListViewApp());
}

class StaticListViewApp extends StatelessWidget {
  const StaticListViewApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('статический ListView')),
        body: ListView(
          padding: EdgeInsets.all(16.0),
          children: [
            ListTile(title: Text('Элемент 1')),
            ListTile(title: Text('Элемент 2')),
            ListTile(title: Text('Элемент 3')),
            ListTile(title: Text('Элемент 4')),
            ListTile(title: Text('Элемент 5')),
          ],
        ), // ListView
      ), // Scaffold
    ); // MaterialApp
  }
}
```



динамический ListView

lib > main.dart > DynamicListViewApp

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(DynamicListViewApp());
}

class DynamicListViewApp extends StatelessWidget {
  const DynamicListViewApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('динамический ListView')),
        body: DynamicListView(),
      ), // Scaffold
    ); // MaterialApp
  }
}

class DynamicListView extends StatelessWidget {
  final List<String> items = List.generate(20, (index) => 'элемент ${index + 1}');

  DynamicListView({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      padding: EdgeInsets.all(16.0),
      itemCount: items.length,
      itemBuilder: (context, index) {
        return ListTile(title: Text(items[index]));
      },
    ); // ListView.builder
  }
}
```

